HS PF

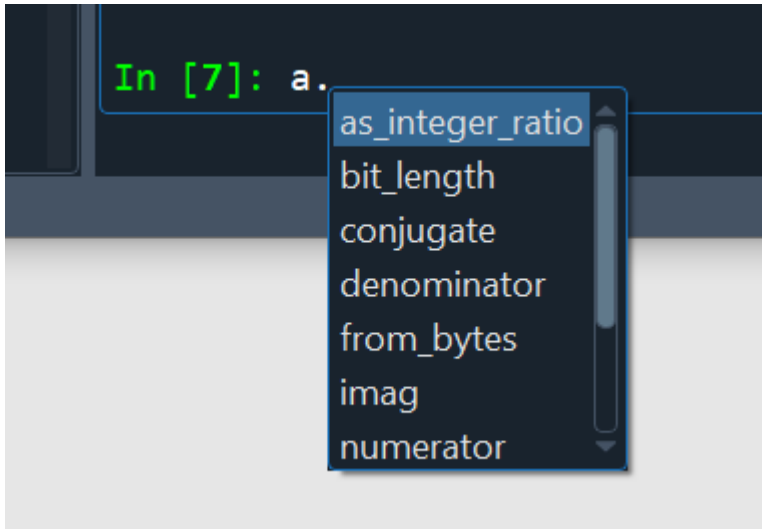
Programmieren in Python

Objektorientierung

Prof. Dr.-Ing. Stefan Kray

Objektorientierung

- Python ist vollständig objektorientiert
- selbst einfache Datentypen wie INT, Bool, ... sind Objekte!
- a=3 → Int
- Die Variable "a" hat Methoden! → a ist Instanz der Klasse <int>



Objektorientierung: Umsetzung in Python (1)

- Objektorientierung basiert auf Klassen
- Von einer Klasse kann eine oder mehrere Instanzen erzeugt werden, die die gemeinsamen Methoden und Attribute übernehmen

```
class Test:
    pass
```

- → Klasse Test (ohne Inhalt)
- Befehl "pass": Leerer Befehl, keine Aktion

```
t=Test()
```

→ t ist jetzt Instanz der Klasse Test

Objektorientierung: Umsetzung in Python (2)

- Instanzen in Python können dynamisch erweitert werden

```
t=Test()
t.a=5
t.b="Hallo"
t.f=lambda x:print("Wert:",x)

t.f(t.a)
```

→ Wert: 5

Konstruktor (1)

- Wenn eine Instanz erzeugt wird, wird der Konstruktor der Klasse aufgerufen (sofern es einen gibt)

```
class Test2:
    def __init__(self):
        self.a=5

t2=Test2()
print(t2.a)    → 5
```

Der Konstruktur heißt "__init__". Üblich ist der Parameter "self", der in c++ dem Parameter "this" entspricht.

"self" bezieht sich jeweils auf ein Attribut der Instanz

self.x erzeugt ein Attribut "x".

Konstruktor (2)

- Wenn eine Instanz erzeugt wird, wird der Konstruktor der Klasse aufgerufen (sofern es einen gibt)

```
class Test2:
    def __init__(self):
        self.a=5

t2=Test2()
t3=Test2()
t2.a=5
t3.a=7

print(t2.a) → 5
print(t3.a) → 7
```

Methoden / Funktionen in Objekten (1)

- Analog zum Konstruktor lassen sich Methoden in Klassen erzeugen

```
class KlasseMitMethoden():
    def __init__(self):
        self.a=5
        self.b="Hallo"
        self.count=0

    def counter(self):
        self.count=self.count+1

    def print(self):
        print("Wert a=",self.a)
        print("Wert b=",self.b)
        print("Wert cnt=",self.count)
```

Methoden / Funktionen in Objekten (2)

- Analog zum Konstruktor lassen sich Methoden in Klassen erzeugen
- Beispiel:

```
instanz1=KlasseMitMethoden()
instanz2=KlasseMitMethoden()
instanz1.counter()
instanz1.counter()
instanz1.counter()
instanz2.counter()
instanz2.counter()
print("instanz1:")
instanz1.print()
print("instanz2:")
instanz2.print()
```

```
class KlasseMitMethoden():
    def __init__(self):
        self.a=5
        self.b="Hallo"
        self.count=0

    def counter(self):
        self.count=self.count+1

    def print(self):
        print("Wert a=",self.a)
        print("Wert b=",self.b)
        print("Wert cnt=",self.count)
```

Destruktor

- __del__ wird beim Löschen der Instanz aufgerufen
- Beispiel:

```
instanz1=KlasseMitMethoden()
instanz2=KlasseMitMethoden()
print("instanz1:")
instanz1.print()
print("instanz2:")
instanz2.print()
instanz2=None
```

→

instanz1:

Wert a= 5

Wert b= Hallo

Wert cnt= 0

instanz2:

Wert a= 5

Wert b= Hallo

Wert cnt= 0

Klasse Gelöscht. Ciao!

```
class KlasseMitMethoden():
    def __init__(self):
        self.a=5
        self.b="Hallo"
        self.cnt=0

    def counter(self):
        self.cnt=self.cnt+1

    def print(self):
        print("Wert a=",self.a)
        print("Wert b=",self.b)
        print("Wert cnt=",self.cnt)

    def __del__(self):
        print("Klasse Gelöscht. Ciao!")
```

Aufgabe fahrzeug.py (1)

- Schreiben Sie eine Klasse Fahrzeug mit den Attributen v (Geschwindigkeit) und s (Strecke) und folgenden Methoden:
  - Methode print:

```
def print(self,text):
        print(text)
        print("Geschwindigkeit:",self.v)
        print("Weg:",self.s)
```

  - Konstruktor: Eingangsparameter v, $self.v = v$ setzen; $self.s = 0$ setzen
  - Methode $fahren$: Eingangsparameter t
    - berechnet die Strecke $self.s$ nach der Zeit t
  - Methode $beschleunigen$: Eingangsparameter a (Beschleunigung), t
    - berechnet die Strecke $self.s$ nach der Zeit t mit der Beschleunigung a
    - berechnet die neue Geschwindigkeit $self.v$ nach der Zeit t mit der Beschleunigung a

gleichförmige Bewegung:

$$s \mathrel{+}= v_0 \cdot t$$
$$v_0 = const$$

beschleunigte Bewegung:

$$s \mathrel{+}= v_0 \cdot t + \frac{1}{2} a t^2$$
$$v \mathrel{+}= a \cdot t$$

```
Testen mit:
auto=Fahrzeug(v=10)
auto.fahren(15)
auto.print("--- nach 15 s ---")
auto.beschleunigen(-1,5)
auto.print("--- nach 20 s ---")
```

Aufgabe fahrzeug.py (2)

- Testen Sie mit:

```
auto=Fahrzeug(v=10)
auto.fahren(15)
auto.print("--- nach 15 s ---")
auto.beschleunigen(-1,5)
auto.print("--- nach 20 s ---")
```

Vererbung

- Ein wichtiger Punkt in der Objektorientierung ist die Vererbung
- Vererbung dient der Wiederverwendung von Gemeinsamkeiten
- Methoden und Attribute einer übergeordneten Klasse können an die Erben weitergegeben werden

Vererbung in Python (1)

- In Python wird die Elternklasse mit Klammern initialisiert:

```
class Auto(Fahrzeug):
    def __init__(self,v):
        pass

auto=Auto()
```

auto

Out[10]: <__main__.Auto at 0x275448f1fa0>

auto ist Instanz der Klasse Auto

Vererbung in Python (2)

- In Python wird die Elternklasse mit Klammern initialisiert:

```
class Auto(Fahrzeug):
    def __init__(self,v):
        pass

auto=Auto()
```

Problem: die Attribute sind nicht enthalten! Warum?

auto.v

Traceback (most recent call last):

Input In [12] in <cell line: 1>

auto.v

AttributeError: 'Auto' object has no attribute 'v'

Vererbung in Python (3)

- Konstruktor der Elternklasse aufrufen

```
class Auto(Fahrzeug):
    def __init__(self,v):
        super().__init__(v)

auto=Auto(10)
```

Funktion super() liefert Elternklasse

super().__init__ ist der Konstruktor der Elternklasse

auto.v

Out[25]: 10

Aufgabe Vererbung.py

- Erweitern Sie die Klasse Auto um eine Funktion "Hupen"
- Schreiben Sie eine Klasse Fahrrad, die die Funktionalität von Fahrzeug hat. Fügen Sie eine Funktion "Klingeln" hinzu
- Schreiben Sie eine Klasse KaputtesAuto, die von Fahrzeug erbt.
  Allerdings kann das Auto nicht Fahren oder Beschleunigen, d. h. Sie brauchen neue Methoden "Fahren", "Beschleunigen", die die alten überschreiben.
  Fügen Sie auch eine Methode "Hupen" hinzu, die "kein Ton" ausgibt.

- Testen Sie mit:

```
auto=Auto(10)
fahrrad=Fahrrad(1)
kaputtesauto=KaputtesAuto(10)
kaputtesauto.fahren(10)
kaputtesauto.print("Kaputt")
```

Überladen

- Im letzten Beispiel haben wir gesehen, dass Methoden überladen werden können
- Die Methode "fahren" in KaputtesAuto wurde durch eine neue Methode ersetzt
- Mit dem Befehl __mro__ zeigt man die Aufrufhierarchie einer Klasse:
  - `KaputtesAuto.__mro__`
    (__main__.KaputtesAuto, __main__.Fahrzeug, object)
  - KaputtesAuto→Fahrzeug→Objekt (Python Oberklasse, von dem jedes Objekt erbt)

Mehrfachvererbung

- Python erlaubt, von mehreren Objekten zu erben
- Dazu werden beide "Elternteile" angegeben:

```
class Test(Auto,KaputtesAuto):
    def __init__(self,v):
        super().__init__(v)

class Test2(KaputtesAuto,Auto):
    def __init__(self,v):
        super().__init__(v)
```

Was gibt folgender Code aus:

```
test=Test(10)
test2=Test2(10)
test.hupen()
test2.hupen()
test.fahren(10)
test.print("--- test ---")
```

```
Test.__mro__
Out[8]:
(__main__.Test,
 __main__.Auto,
 __main__.KaputtesAuto,
 __main__.Fahrzeug,
 object)

Test2.__mro__
Out[9]:
(__main__.Test2,
 __main__.KaputtesAuto,
 __main__.Auto,
 __main__.Fahrzeug,
 object)
```